

Sockets in C

Operating Systems and Networks, M26

Taken by Sami and Advait

The Shape of a Socket Program

The first two steps are identical. Everything after that depends on the protocol you picked.

01

Create a socket

You get a file descriptor



02

Describe an address

An IP and a port in a struct



TCP

Connect first, then send

UDP

Send straight away

Step 1

Create a Socket

Same first call for both protocols. The second argument is the only difference.

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0); // TCP
int sockfd = socket(AF_INET, SOCK_DGRAM, 0);  // UDP
```

AF_INET selects IPv4 addressing. For IPv6, use **AF_INET6** and a `sockaddr_in6` struct instead.

SOCK_STREAM and **SOCK_DGRAM** set the protocol: a continuous byte stream for TCP, or independent packets for UDP.

The return value is a file descriptor, an integer the kernel uses to track this socket. Nothing is connected yet, and no address is claimed.

Step 2

Describe an Address

A struct holding an IP and a port. Also the same for both protocols.

```
struct sockaddr_in addr;  
memset(&addr, 0, sizeof(addr));  
addr.sin_family = AF_INET;  
addr.sin_port = htons(8080);  
addr.sin_addr.s_addr = INADDR_ANY;
```

`htons` puts the port in network byte order.

`INADDR_ANY` tells a server to listen on any IP on the machine.

A client points at one specific server instead:

```
inet_pton(AF_INET, "127.0.0.1",  
          &addr.sin_addr);
```

The socket functions take a generic `sockaddr` pointer, so `sockaddr_in` is cast to it whenever you call `bind` or `connect`.

Step 2, continued

Why htons Exists

CPUs do not all store multi-byte numbers the same way. Network protocols pick one order, so every machine has to convert into it.

Host byte order

The order your CPU uses internally. On most desktop and server chips today, this is little-endian: the least significant byte comes first.

Network byte order

The order TCP/IP always uses on the wire: big-endian, most significant byte first. Every machine has to agree on this, regardless of its own CPU.

Port 8080 stored little-endian

90

byte 0 (lowest address)

1F

byte 1

Least significant byte first. This is how the number sits in memory on most desktop and server CPUs.

Port 8080 stored big-endian

1F

byte 0 (lowest address)

90

byte 1

Most significant byte first. This is the order network byte order always uses, on every machine.

Step 2, continued

htons and ntohs

Four conversion functions exist. These two cover the port number in a `sockaddr_in`.

`htons(port)`

host to network short

Converts a 16 bit number from your CPU's byte order into network byte order. Use this when filling in `sin_port`.

`ntohs(port)`

network to host short

Converts back the other way. Use this when reading a port out of a `sockaddr_in` you received, before printing or comparing it.

On a big-endian machine both functions do nothing. Calling them anyway costs nothing and keeps the code portable, so call them always.

TCP

TCP Server Setup

```
bind(sockfd, (struct sockaddr*)&addr, sizeof(addr));  
listen(sockfd, 10);  
int client_fd = accept(sockfd, NULL, NULL);
```

bind

Claims that address and port for this socket.

listen

Marks it as passive and queues up to 10 pending connections.

accept

Blocks until a client arrives, then returns a new socket for that client.

The 10 passed to listen is the backlog, the number of pending connections the kernel will queue before you call accept. The original sockfd keeps listening, so you can call accept again for the next client.

TCP

Client Connects, Server Accepts

Server

socket()

bind()

listen()

accept() waiting

connection



Client

socket()

connect()

No bind needed. The OS picks a local port for you.

```
connect(sockfd, (struct sockaddr*)&addr, sizeof(addr));
```

TCP

Sending Over TCP

The socket already knows who it is talking to, so the calls only need a buffer.

```
send(sockfd, buf, len, 0);
```

On the server use `client_fd`, the descriptor `accept` gave you.
On the client use `sockfd`.

```
recv(sockfd, buf, sizeof(buf), 0);
```

Both calls return how many bytes actually moved, which can be fewer than you asked for. Real code loops until it has everything.

UDP

UDP Skips the Connection

Server

```
bind(sockfd,  
      (struct sockaddr*)&addr,  
      sizeof(addr));
```

That is the whole setup. Bind so packets sent to that port reach you.

Client

nothing

No bind, no connect. Go straight to sending, with the server address attached to each packet.

There is no `listen` and no `accept`, because UDP does not track connections. Every packet is handled on its own, with no memory of what came before it.

UDP

Sending Over UDP

Every call carries an address, since the socket has no fixed peer.

```
sendto(sockfd, buf, len, 0,  
        (struct sockaddr*)&addr,  
        sizeof(addr));
```

`sendto` names the destination on every packet.

```
struct sockaddr_in from;  
socklen_t from_len = sizeof(from);  
recvfrom(sockfd, buf, sizeof(buf), 0,  
          (struct sockaddr*)&from,  
          &from_len);
```

`recvfrom` fills in `from` so you know who to reply to.

UDP

Broadcasting to Every Host

A UDP packet can be sent to every host on the local network at once, instead of to one address.

```
int enable = 1;
setsockopt(sockfd, SOL_SOCKET, SO_BROADCAST, &enable, sizeof(enable));

addr.sin_addr.s_addr = INADDR_BROADCAST; // 255.255.255.255
```

`SO_BROADCAST` must be enabled on the socket before you are allowed to send to a broadcast address. Without it, the send fails.

`INADDR_BROADCAST` is the address 255.255.255.255, meaning every host on the local network. The `sendto` call itself is unchanged.

UDP

Using It to Find a Server

A client that does not know the server's address yet can broadcast a request and wait for a reply.

```
struct timeval tv = {1, 0};
setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

while (1) {
    sendto(sockfd, &pkt, sizeof(pkt), 0, (struct sockaddr*)&bcast_addr, sizeof(bcast_addr));
    ssize_t n = recvfrom(sockfd, &reply, sizeof(reply), 0, (struct sockaddr*)&from, &from_len);

    if (n > 0) break; // got a reply
    // timed out, broadcast again
}
```

`SO_RCVTIMEO` puts a limit on how long `recvfrom` will block, so the client can retry the broadcast if nobody answers yet.

The reply arrives through `recvfrom`, which fills in `from`: the server's real address. From here on, the client sends directly to that address instead of broadcasting again.

Side by Side

Step	TCP	UDP
Create	<code>socket(..., SOCK_STREAM, 0)</code>	<code>socket(..., SOCK_DGRAM, 0)</code>
Server setup	<code>bind, listen, accept</code>	<code>bind</code>
Client setup	<code>connect</code>	<code>nothing</code>
Send	<code>send()</code>	<code>sendto()</code>
Receive	<code>recv()</code>	<code>recvfrom()</code>

Choosing Between Them

TCP

Ordered, reliable byte stream

The kernel retransmits lost data and keeps bytes in order.
You pay for it with setup time and buffering.

Good for HTTP, databases, file transfer.

UDP

Individual packets, no guarantees

Packets can arrive late, out of order, or not at all. Nothing waits, so latency stays low.

Good for DNS, game state, live audio and video.

Start with TCP unless you have a specific reason not to. Both also work over IPv6, using `AF_INET6` in place of `AF_INET`.